KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

# 함수 1
# 정의와 return

자주 쓰는 계산에 이름을 붙이고
값을 넣어 결과를 돌려받아요

```
def calc_total(cart, menu):
    total = 0
    for m in cart:
        total += menu[m]
    return total

calc_total(cart, menu)
```

# 오늘의 순서


| | | | |
|---|---|---|---|
| 1 | 왜 필요한가 | 복사한 계산, 재료와 결과 상자, 가장 짧은 def | 4–6쪽 |
| 2 | 정의와 호출 | def 줄 다섯 조각, 정의만 하면 안 찍혀요, 실행 순서 | 7–9쪽 |
| 3 | 매개변수와 인자 | 재료 칸 하나, 여러 개, 순서대로 들어가요 | 10–12쪽 |
| 4 | return | 돌려주기, print와 비교, None, 바로 끝, 값 두 개 | 13–17쪽 |
| 5 | 오류와 함정 | 괄호 빼기, 인자 수, return 아래 줄, None 계산, NameError | 18–22쪽 |
| 6 | 실습 | 장바구니 총액, 품절 확인, 성적 등급, 할인 결제 | 23–26쪽 |


3쪽은 노트북 준비, 27–28쪽은 막힐 때와 한 장 요약이에요.

들어가기

# 18에서 이어서 해요

- hello-uv 노트북, 커널 .venv 09
- 변수 이름 규칙, Restart 10
- f-string, 리스트, for, 튜플 꺼내 담기 13
- 딕셔너리 menu["latte"], None 15
- if, elif, else 17
- range, while, break 18

**기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.**

## 노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

# 같은 계산을 손님마다 복사하고 있어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = ["coffee", "cake"]
total = 0
for m in cart:
    total += menu[m]
print(f"합계 {total:,}원")
cart = ["latte", "latte", "cake"]
total = 0
for m in cart:
    total += menu[m]
print(f"합계 {total:,}원")
```

Python

```
합계 11,000원
합계 16,500원
```

← 첫 손님 합계 4줄

← 둘째 손님, 글자까지 같은 4줄

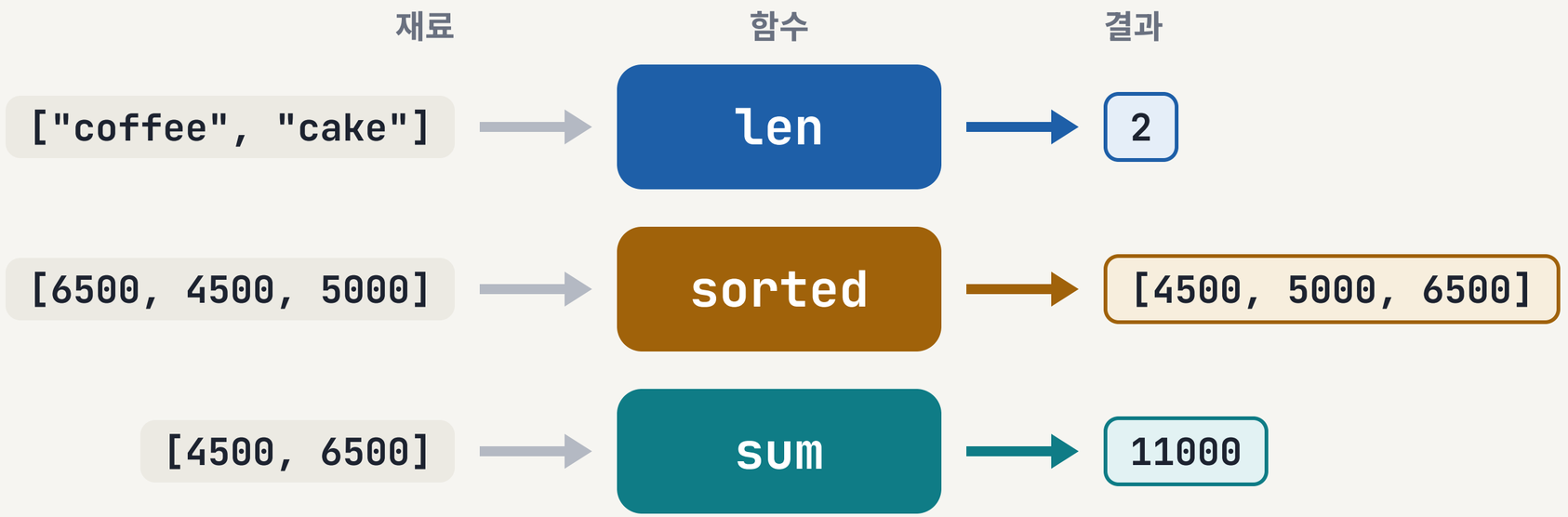
셋째 손님이 오면 또 붙여 넣어요

할인이 생기면 전부 찾아 고쳐요

**같은 일을 하는 줄에 이름을 붙이고, 이름으로 불러 쓰는 게 함수예요.**

1. 왜 필요한가

# 함수는 재료를 넣으면 결과가 나오는 상자예요




[2]
```python
cart = ["coffee", "cake"]
print(len(cart))
print(sorted([6500, 4500, 5000]))
print(sum([4500, 6500]))
```

```
2
[4500, 5000, 6500]
11000
```

← 13에서 써 본 상자들이에요

오늘은 이런 상자를 def로 만들어요

1. 왜 필요한가

# def로 만들고, 이름()으로 불러요

[3]
```python
def hello():
    print("어서 오세요")


hello()
hello()
```

```
어서 오세요
어서 오세요
```

← 정의: 만들어 두기

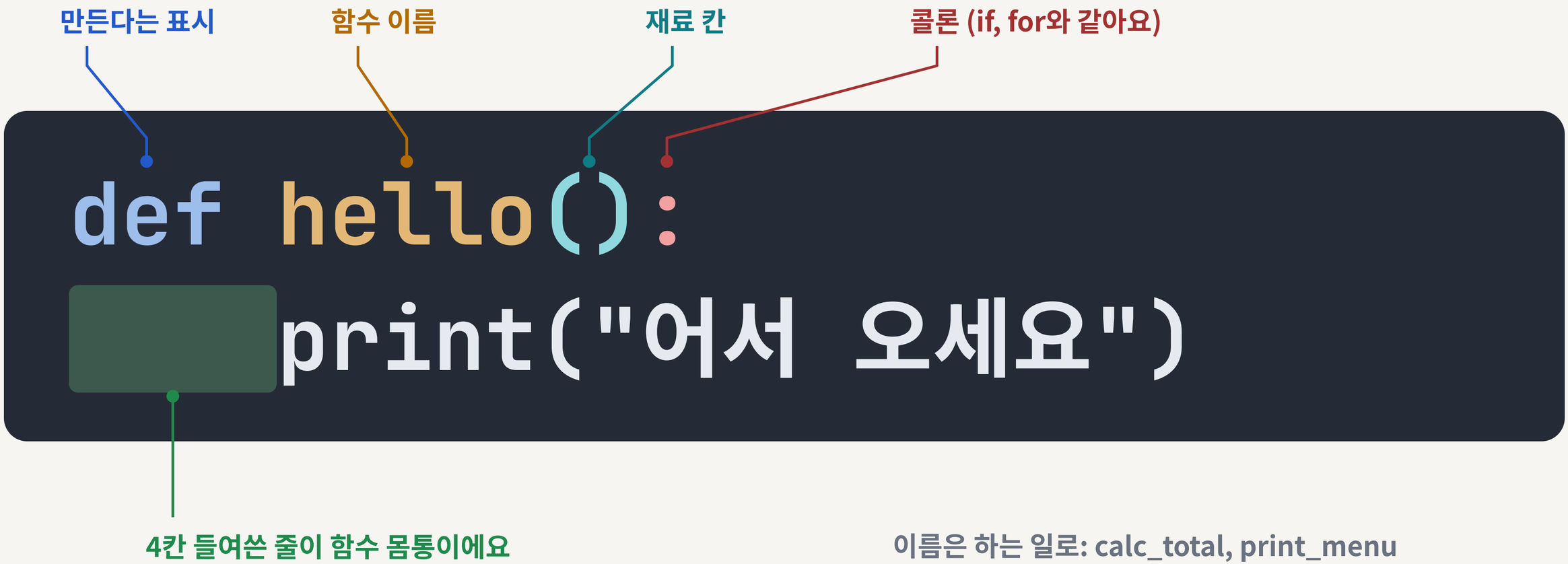
← 호출: 실행하기

두 번 불러서 두 번 찍혀요

**몸통은 한 번만 적고, 부른 만큼 실행돼요.**

2. 정의와 호출

# def 줄은 다섯 조각으로 읽어요




이름은 하는 일로: calc_total, print_menu

**모양은 if, for와 같아요. 콜론 뒤 4칸 들여쓴 줄이 한 덩어리예요.**

2. 정의와 호출

# 정의만 하면 아무것도 찍히지 않아요

[1]
```python
def hello():
    print("어서 오세요")
    print("주문하시겠어요?")
```

← hello라는 이름에 몸통을 기억만 해요

출력 없음

[ ]
```python
hello()
```

Shift + Enter

어서 오세요
주문하시겠어요?

← 부르면 그제야 실행돼요

**def는 기억해 두기만 하고, 실행은 이름()을 부를 때 해요.**

2. 정의와 호출

# 부르면 몸통으로 갔다가 제자리로 돌아와요

앞 셀에서 정의한 hello

[1]
```
def hello():
    print("어서 오세요")
    print("주문하시겠어요?")
```
Python

이번에 실행하는 셀

[3]
```
print("가게 문 열어요")
hello()
print("다음 손님")
hello()
```
Python

출력

1 가게 문 열어요
2 어서 오세요
2 주문하시겠어요?
3 다음 손님
4 어서 오세요
4 주문하시겠어요?

**몸통이 끝나면 부른 줄의 다음 줄로 돌아와요**

**부를 때마다 몸통을 처음부터 끝까지 실행하고 돌아와요.**

3. 매개변수와 인자

# 괄호 안에 재료 칸을 만들어요

[1]
```python
def greet(name):
    print(f"{name}님, 어서 오세요")

greet("kim")
greet("lee")
```

```
kim님, 어서 오세요
lee님, 어서 오세요
```




greet("kim")를 부를 때




greet("lee")를 부를 때

**name 매개변수, def 괄호 안의 칸 이름**

**"kim" 인자, 부를 때 괄호에 넣는 값**

# 재료 칸은 쉼표로 여러 개 만들어요

[2]
```python
def order(menu_name, qty):
    print(f"{menu_name} {qty}잔 주문 받았어요")

order("latte", 2)
order("coffee", 1)
```

```
latte 2잔 주문 받았어요
coffee 1잔 주문 받았어요
```

**menu_name**

"latte"

**qty**

2

첫째 값은 첫째 칸에
둘째 값은 둘째 칸에

칸이 여러 개면 쉼표로 나눠 적고, 부를 때도 쉼표로 나눠 넣어요.

3. 매개변수와 인자

# 인자는 적은 순서대로 칸에 들어가요

```
[3]  order(2, "latte")
                                                    Python
```

2 latte잔 주문 받았어요 ← 오류 없이 이상하게 찍혀요

```
def order(menu_name, qty):   ← def 줄의 칸 순서를 보고 넣어요
```

```
order("latte", 2)
```
← 값을 칸 순서대로 바꿔 넣어요

**칸 순서를 바꿔 넣으면 값도 바뀌어 들어가요.**



4. return

# return은 결과를 부른 자리로 돌려줘요

[1]
```python
def price_of(qty):
    return 4500 * qty

x = price_of(2)
print(x)
print(price_of(3) + 1000)
```
돌려줄 값

Python

```
9000
14500
```

부른 자리가 값으로 바뀌어요

```
x = 9000
```

```
13500 + 1000 = 14500
```

**return 뒤의 값이 함수를 부른 자리에 들어가요.**

4. return

# print는 화면에 찍기만, return은 값을 돌려줘요

## print로 찍는 함수

[2]
```python
def show(qty):
    print(4500 * qty)

a = show(2)
print("a:", a)
```

```
9000
a: None
```

9000은 찍혔을 뿐, a에는 None

## return으로 돌려주는 함수

[3]
```python
def calc(qty):
    return 4500 * qty

b = calc(2)
print("b:", b)
print(b + 1000)
```

```
b: 9000
10000
```

b에 9000이 담겨서 더할 수 있어요

**사람이 볼 값은 print, 다음 계산에 쓸 값은 return이에요.**

4. return

# return이 없으면 None을 돌려줘요

[4]
```python
result = print("안녕")
print(result)
print(type(result))
```

```
안녕
None
<class 'NoneType'>
```

← print도 돌려주는 값이 없어요

← None은 '값이 없음', 15의 get과 같아요

[5]
```python
def hi():
    print("안녕")
print(hi())
```

```
안녕
None
```

← 직접 만든 함수도 return이 없으면 None

4. return

# return을 만나면 함수가 바로 끝나요

[6]
```python
def check(stock):
    if stock == 0:
        return "품절"
    return "주문 가능"

print(check(0))
print(check(3))
```

```
품절
주문 가능
```

← check(0)은 여기서 끝

← check(0)은 여기까지 안 와요

| 부르기 | stock == 0 | 만난 return |
|---|---|---|
| `check(0)` | `True` | `return "품절"` |
| `check(3)` | `False` | `return "주문 가능"` |

**return은 값을 돌려주면서 함수를 끝내요. 그래서 else가 없어도 돼요.**

4. return

# 쉼표로 값 여러 개를 한 번에 돌려줘요

[7]
```python
def summary(prices):
    return sum(prices), len(prices)

print(summary([4500, 5000, 6500]))
total, n = summary([4500, 5000, 6500])
print(total, n)
```

Python

```
(16000, 3)
16000 3
```

← 괄호로 묶인 튜플 하나로 와요 (13)

| total | n |
|---|---|
| 16000 | 3 |

**13의 꺼내 담기와 같아요**

왼쪽 이름 수 = 돌려준 값 수

**return a, b는 튜플로 돌려주고, x, y = 함수()로 나눠 받아요.**

5. 오류와 함정

# 괄호를 빼면 실행하지 않고 함수 자체가 나와요

[1]
```python
def hello():
    print("어서 오세요")

hello
```

```
<function __main__.hello()>
```
← 실행하지 않고 함수만 보여 줘요

[2]
```python
print(hello)
```

```
<function hello at 0x7efc68143530>
```
← 0x 뒤 숫자는 실행마다 달라요

`hello()` ← 괄호를 붙여야 실행돼요

**부를 때는 재료가 없어도 괄호 ()를 꼭 붙여요.**

5. 오류와 함정

# 칸 수와 인자 수가 다르면 TypeError가 나요

[3]
```python
def order(menu_name, qty):
    print(f"{menu_name} {qty}잔 주문 받았어요")

order("latte")
```

```
TypeError: order() missing 1 required positional argument: 'qty'
```

← qty 칸이 비었어요

[4]
```python
order("latte", 2, "L")
```

```
TypeError: order() takes 2 positional arguments but 3 were given
```

← 칸은 2개, 넣은 값은 3개

× ~~order("latte")~~ × ~~order("latte", 2, "L")~~ → ✓ `order("latte", 2)`

**def 줄의 칸 수만큼 인자를 넣어요.**

# return 아래에 적은 줄은 실행되지 않아요

[5]
```python
def calc(qty):
    total = 4500 * qty
    return total
    print("계산 끝")

print(calc(2))
```

← 여기서 함수를 나가요

← 그래서 이 줄은 안 와요

```
9000
```

← 계산 끝이 없는데 오류도 없어요

print를 return 위로 옮겨요

[ ]
```python
def calc(qty):
    total = 4500 * qty
    print("계산 끝")
    return total
```

**함수에서 할 일을 다 적고 return은 마지막에 둬요.**

5. 오류와 함정

# 찍기만 하는 함수로 계산하면 TypeError가 나요

[6]
```
def show(qty):
    print(4500 * qty)

show(2) + 1000
```
Python

```
9000
```

```
TypeError: unsupported operand type(s) for +: 'NoneType' and 'int'
```

`None + 1000` ← show(2)가 돌려준 None으로 바뀌어요

`return 4500 * qty` ← 계산에 쓸 값은 return으로

**NoneType이 보이면 return이 빠진 함수를 찾아요.**

5. 오류와 함정

# 정의보다 먼저 부르면 NameError가 나요

[1]
```
welcome()

def welcome():
    print("어서 오세요")
```
Python

```
NameError: name 'welcome' is not defined
```

← 아직 def를 만나기 전이에요

← 오류에서 멈춰서 여기까지 못 와요

[2]
```
welcome()
```
Python

```
NameError: name 'welcome' is not defined
```

← 다시 불러도 같은 오류

basics.ipynb ▷ Run All ↻ Restart Clear All Outputs

← def 셀을 지워도 커널은 기억해요

Restart 뒤 위에서부터 실행하면 드러나요

**def 셀을 먼저 실행하고, 부르는 셀은 아래에 둬요.**

6. 실습

# 장바구니 총액을 함수로 만들어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}

def calc_total(cart, menu):
    total = 0
    for m in cart:
        total += menu[m]
    return total

cart1 = ["coffee", "cake"]
cart2 = ["latte", "latte", "cake"]
print(f"합계 {calc_total(cart1, menu):,}원")
print(f"합계 {calc_total(cart2, menu):,}원")
```

Python

← 할인이 생기면 여기만 고쳐요

← 1파트에서 복사하던 4줄

← 손님마다 한 줄

```
합계 11,000원
합계 16,500원
```

**반복해서 쓰는 계산은 함수로 묶고, return으로 결과를 돌려받아요.**

6. 실습

# 주문할 수 있는지 True, False로 돌려줘요

[2]
```python
def can_order(stock, name):
    if stock[name] > 0:
        return True
    return False

stock = {"coffee": 3, "latte": 0, "cake": 2}
for o in ["coffee", "latte", "cake"]:
    if can_order(stock, o):                    주문할 수 있나요?
        print(o, "주문 받았어요")
    else:
        print(o, "품절이에요")
```
Python

```
coffee 주문 받았어요
latte 품절이에요
cake 주문 받았어요
```

| o | stock[o] | 돌려준 값 |
|---|---|---|
| coffee | 3 | True |
| latte | 0 | False |
| cake | 2 | True |

**False라서 else로 가요**

**True, False를 돌려주는 함수는 if 뒤에 바로 써요.**

6. 실습

# 17의 등급 계산을 함수로 바꿔요

[3]

```python
def grade(score):
    if score >= 90:
        return "A"
    elif score >= 80:
        return "B"
    elif score >= 70:
        return "C"
    return "F"

scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
for name, score in scores:
    print(f"{name:<5}{score:>3}  {grade(score)}")
```

Python

```
kim   92  A
lee   78  C
park  85  B
choi  59  F
```

← for 안에는 찍는 줄 하나만 남았어요

← 등급 정하기만 함수로 뺐어요

17에서는 for 안에 이만큼 있었어요

```python
for name, score in scores:
    if score >= 90:
        grade = "A"
    elif score >= 80:
        grade = "B"
    ...
    print(... grade)
```

6. 실습

# 함수 안에서 다른 함수를 불러요

[4]

```python
def discount_rate(total):
    if total >= 20000:
        return 10
    elif total >= 10000:
        return 5
    return 0

def pay(total):
    rate = discount_rate(total)        다른 함수 부르기
    sale = total * rate // 100
    return total - sale

print(pay(8000))
print(pay(15500))
print(pay(22000))
```

Python

```
8000
14725
19800
```

| total | rate | sale | pay() |
|---|---|---|---|
| 8000 | 0 | 0 | 8000 |
| 15500 | 5 | 775 | 14725 |
| 22000 | 10 | 2200 | 19800 |

**15500원은 5% 할인, 775원이 빠져요**

**기준이 바뀌면 discount_rate만 고쳐요**

정리

# 막히면 오류의 마지막 줄을 봐요

| 보이는 것 | 까닭 | 고치기 |
|---|---|---|
| `<function hello at 0x...>` | 괄호를 빼서 실행하지 않았어요 | **`hello()`** |
| `missing 1 required positional argument` | 인자를 덜 넣었어요 | **def 줄 칸 수 세기** |
| `takes 2 positional arguments but 3 were given` | 인자를 더 넣었어요 | **def 줄 칸 수 세기** |
| `NameError: name 'welcome' is not defined` | def 셀보다 먼저 불렀어요 | **def 셀 먼저 실행** |
| **돌려받은 값이 None** | 함수에 return이 없어요 | **print 대신 return** |
| `'NoneType' and 'int'` | None으로 계산했어요 | **return 넣기** |
| **return 아래 print가 안 찍혀요** | return에서 함수가 끝났어요 | **return 위로 옮기기** |

**결과가 이상하면 함수 안에 print를 넣어 받은 값을 찍어 봐요.**

# 한 장 요약

```
def calc(price, qty):
    return price * qty

x = calc(4500, 2)
a, b = summary(prices)
```

← 정의 def 이름(매개변수):, 콜론 뒤 4칸 몸통

← return 값을 돌려주고 바로 끝내요. 없으면 None

← 호출 이름(인자), 인자는 적은 순서대로 칸에

← 값 두 개 return a, b는 튜플, 받을 땐 꺼내 담기

| | 언제 쓰나요 |
|---|---|
| `print` | 사람이 화면에서 볼 값을 찍을 때 |
| `return` | 결과를 변수에 담거나 다음 계산에 쓸 때 |

**다음 챕터 20: 기본값, 키워드 인자, 지역 변수와 전역 변수**